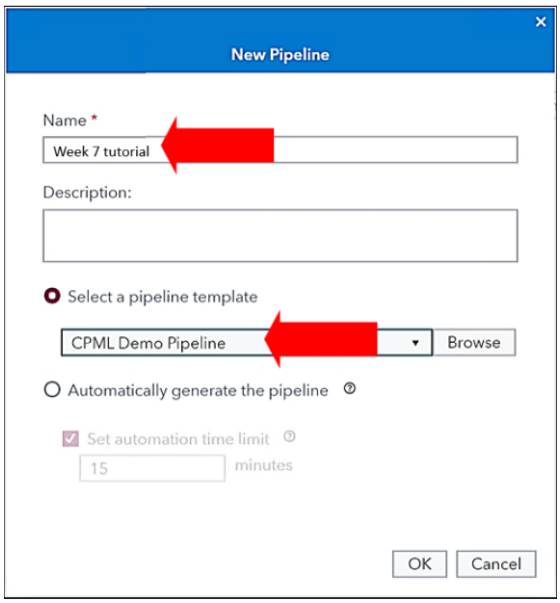
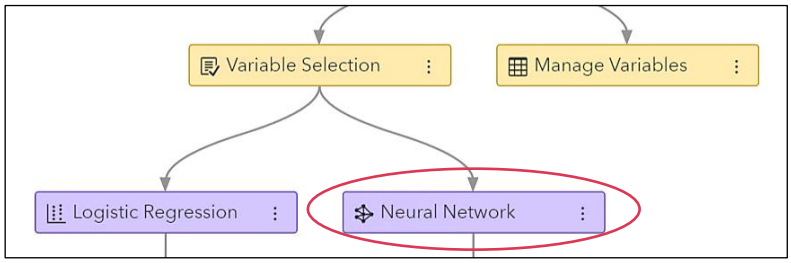
Building a Neural Network Model with Default Settings

In this demonstration, you create a new pipeline using the CPML Demo Pipeline and add a Neural Network node to it. You build the neural network model using the default settings of the node.

1. Click the plus sign (**+**) next to the Week 5 tutorial pipeline tab to add a new pipeline.
2. In the New Pipeline window, enter **Week 7 tutorial** in the **Name** field, access the menu under the **Select a pipeline template** property, and select **CPML Demo Pipeline**.



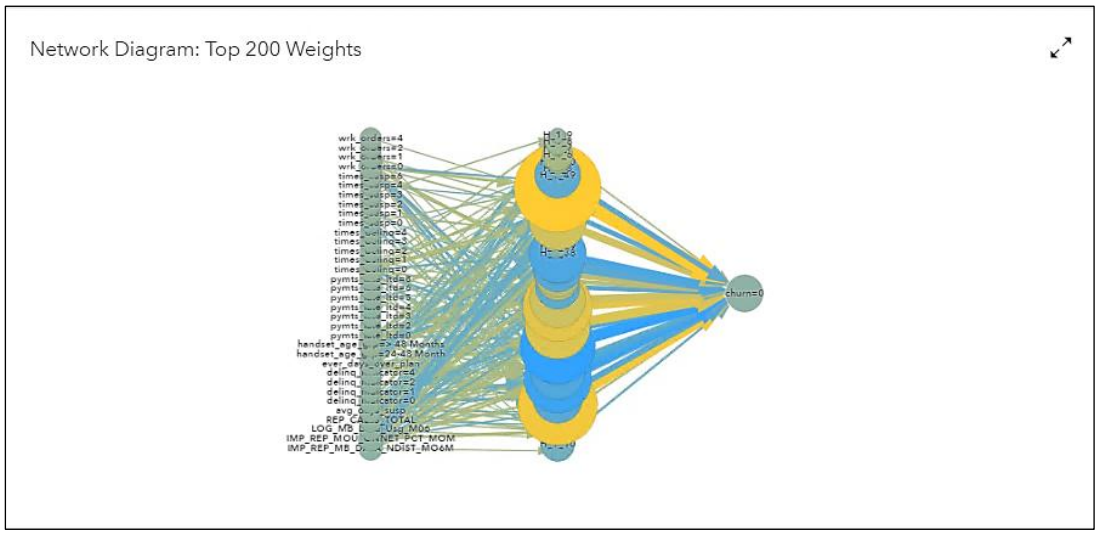
3. Click **OK**.
4. In the Lesson 4 pipeline, right-click the **Variable Selection** node and select **Add child node** ⇨ **Supervised Learning** ⇨ **Neural Network**.



5. Keep all properties for the neural network at their defaults.
6. Run the **Neural Network** node.

7. Open the results for the node.

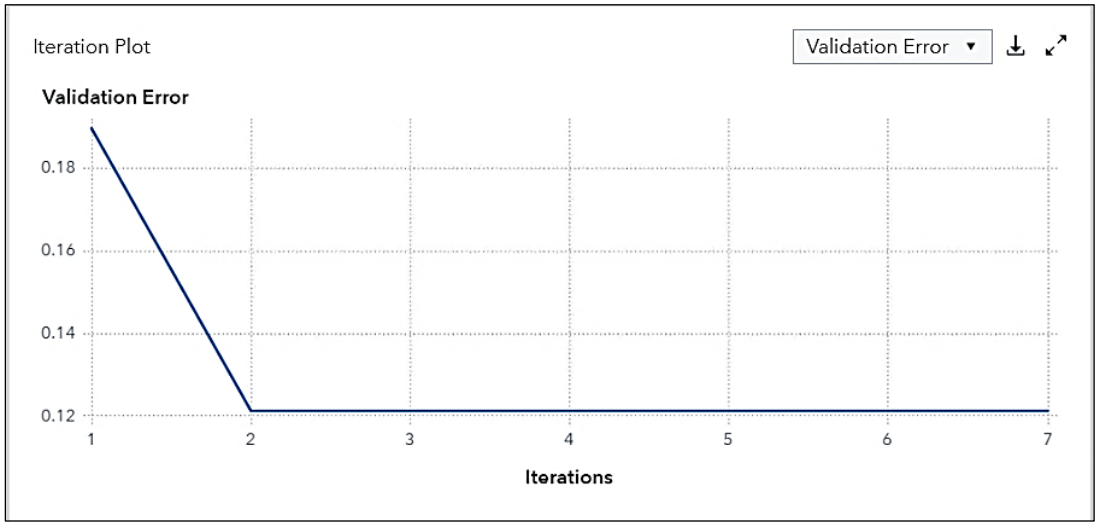
   There are several charts and plots to help you evaluate the model's performance. The first plot is *Network Diagram: Top 200 Weights*, which presents the final neural network structure for this model, including the hidden layer and the hidden units.



   Network Diagram: Top 200 Weights displays a diagram of the neural network when there are fewer than six hidden layers in the network. Only the links that correspond to the top 200 weights are displayed. You can interactively control the range of these weights to display.

   The Network diagram displays the input nodes, hidden nodes, connections, and output nodes of a neural network. Nodes are represented as circles, and links between the nodes are lines connecting two circles. The size of the circle represents the magnitude of the absolute value of that node, relative to the model, and the color indicates whether that value is positive or negative. Similarly, the size of the line between two nodes indicates the strength of the link, and the color indicates whether that value is positive or negative.

   The *Iteration plot* shows the model's performance based on the valid error throughout the training process when new iterations are added to achieve the final model.



   The Iteration plot displays a line graph of the validation error, if reported, as a function of the epoch. To examine the loss or objective as a function of the epoch, use the drop-down menu in the upper right corner.

The *Node Score Code window* shows the final score code that can be deployed in production.

Node Score Code

```
   length _strfmt_ $12; drop _strfmt_;
   _strfmt_ = ' ';

   array _tlevname_42189460_{2} $2 _temporary_ ( ' 1'
   ' 0');

   length I_churn $2;
   array _node_val_42189460_{97} _temporary_;

   _badval_ = 0;
   _dropinput_ =                    1;
   _drop_ =                   1;

   _numval_ = avg_days_susp;
   if missing(_numval_) then do;
```

Similarly, the *Training Code window* shows the train code that can be used to train the model based on different data sets or in different platforms.

Training Code

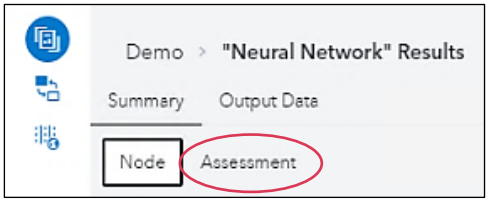
```
*------------------------------------------------------------*;
* Macro Variables for input, output data and files;
  %let dm_datalib =;
  %let dm_lib     = WORK;
  %let dm_folder  = %sysfunc(pathname(work));
*------------------------------------------------------------*;
*------------------------------------------------------------*;
  * Training for neural;
*------------------------------------------------------------*;
*------------------------------------------------------------*;
  * Initializing Variable Macros;
*------------------------------------------------------------*;
%macro dm_unary_input;
%mend dm_unary_input;
%global dm_num_unary_input;
```

Finally, the *Output window* shows the final neural network model parameters, the iteration history, and the optimization process. (Only a portion of the information in the Output window is shown below.)

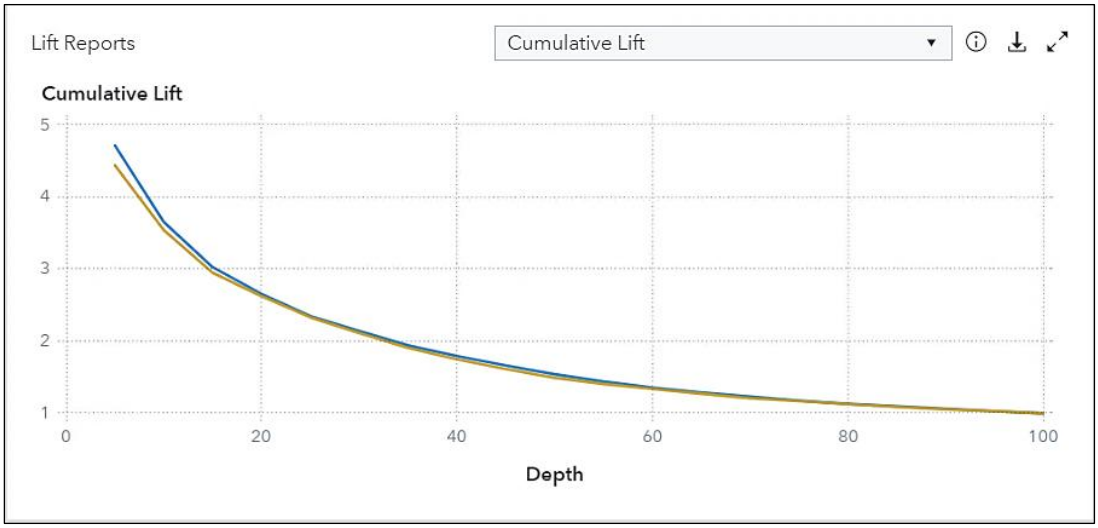
The NNET Procedure

Model Information	
Model	Neural Net
Number of Observations Used	39590
Number of Observations Read	39590
Target/Response Variable	churn
Number of Nodes	97
Number of Input Nodes	45
Number of Output Nodes	2
Number of Hidden Nodes	50
Number of Hidden Layers	1
Number of Weight Parameters	2300
Number of Bias Parameters	52
Architecture	MLP
Seed for Initial Weight	12345
Optimization Technique	LBFGS
Number of Neural Nets	1
Objective Value	1.7563371218
Misclassification Rate for Validation	0.1213

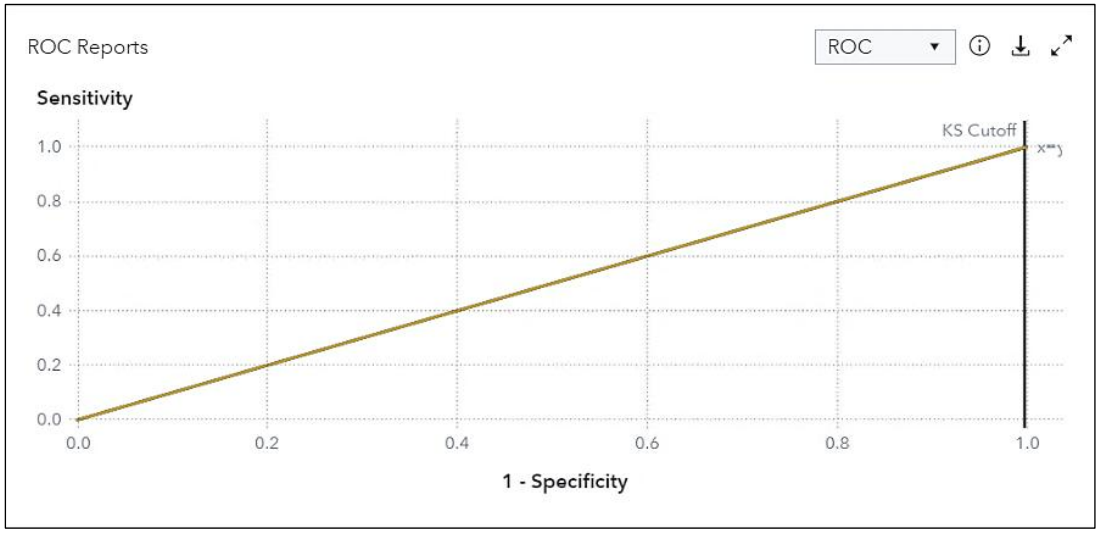
8. Click the **Assessment** tab.



The first chart is the *Lift Report*. By default, it displays the *Cumulative Lift* plot showing the model's performance ordered by the percentage of the population. This chart is very useful for selecting the model based on a particular target of the customer base. It shows how much better the model is than the random events. Other plots, such as for Lift, Gain, and Response Percentage (to name only a few), are available from a menu.



For a binary target, you also have the *ROC Reports*, which show the model's performance considering the true positive rate and the false positive rate. It is good to foresee the performance on a specific business events, when all positive cases are selected. It shows the model's performance based on the positive cases that were predicted right and the negative cases that were predicted wrong. Other plots, such as those for accuracy, are available from a menu.



Finally, you have the *Fit Statistics* table, which shows the model's performance based on some assessment measures such as average squared error. (Only a portion of the table is shown below. You might need to maximize the table to see these values.)

Target …	Data Role	Partitio…	Form…	Sum of …	Averag…
churn	TRAIN	1	1	39,590	0.1237
churn	VALIDATE	0	0	16,967	0.1238

The fit statistics shows an average squared error of 0.1238 on the VALIDATE partition.

9. Close the Results window.

End of Demonstration

Improving a Neural Network Model by Changing the Network Architecture Parameters

In this demonstration, you change the default settings of the Neural Network node in the Lesson 4 pipeline. You modify the network architecture parameters.

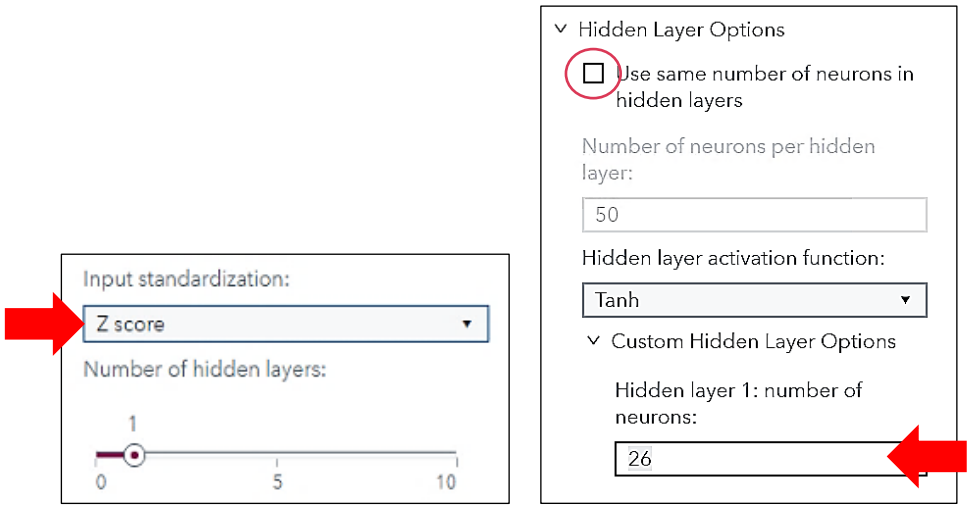
1. Recall that the average squared error of the previous model, based on the default settings, was 0.1238 on the VALIDATE partition.

   Try to improve the neural network performance by changing some of the default settings assigned to the network architecture.

2. Change **Input standardization** from Midrange to **Z score**. In this way, you rescale your data to have a mean of 0 and a standard deviation of 1. Midrange standardization results in the range values with a minimum of -1 and maximum of +1. Midrange standardization is usually applied to the tanh, arctangent, Elliott, sine, and cosine activation functions.

3. Clear the box for **Use the same number of neurons in hidden layers**. This enables you to use a different number of neurons in each hidden layer. However, this setting is redundant here because we are using only one hidden layer.

4. Under **Custom Hidden Layer Options**, enter **26** for **Hidden layer 1: number of neurons** (twice as many as the number of inputs). This specifies the number of hidden neurons in the first hidden layer for a network with one or more hidden layers. Because you have only one hidden layer, this can also be accomplished by directly using the **Number of neurons per hidden layer** property (which is inactive in the image below).



5. Run the **Neural Network** node.

6. Open the results for the node.

7. Click the **Assessment** tab.

Target ...	Data Role	Partitio...	Formatt...	Sum of ...	Averag...
churn	TRAIN	1	1	39,590	0.0690
churn	VALIDATE	0	0	16,967	0.0720

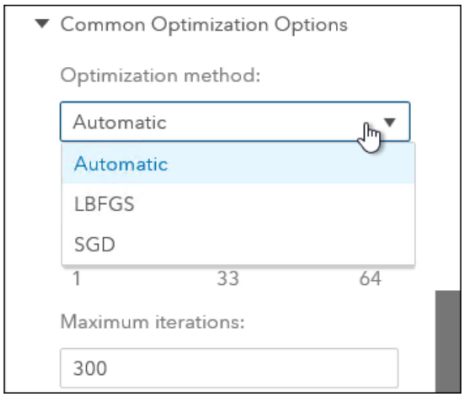
   The average squared error for the tuned neural network model is 0.0720 on the VALIDATE partition. This fit statistic is much better than the first model, which was fit by using the default settings. (But keep in mind results like this are data dependent! We cannot expect such large improvements every time that we try change settings for a model.)

8. Close the Results window.

End of Demonstration

Numerical Optimization Methods in Model Studio

- Limited-Memory Broyden-Fletcher-Goldfarb-Shanno (**LBFGS**)
- Stochastic Gradient Descent (**SGD**)



Copyright © SAS Institute Inc. All rights reserved.

There are two optimization methods available in the Neural Network node of Model Studio: limited memory BFGS and stochastic gradient descent. They are both discussed briefly over the next few slides.

In the Neural Network properties panel, you can select a numerical optimization method under **Common Optimization Options**. The menu for **Optimization method** is shown above. The default setting of **Optimization method** is **Automatic**, which selects one of the two available optimization methods based on the number of hidden layers. LBFGS is used when there are 2 or less hidden layers. SGD is used otherwise.

LBFGS

Limited-Memory Broyden-Fletcher-Goldfarb-Shanno optimization algorithm:

- LBFGS is the default optimization method for the Neural Network node when two or fewer hidden layers are used.
- An estimation of the inverse Hessian matrix is used to steer the search.
- Rather than a full *n*x*n* (*n* = number of variables) approximation to the inverse Hessian, only a few vectors are stored to represent the approximation.
- This method is well suited for optimization problems with a large number of variables.

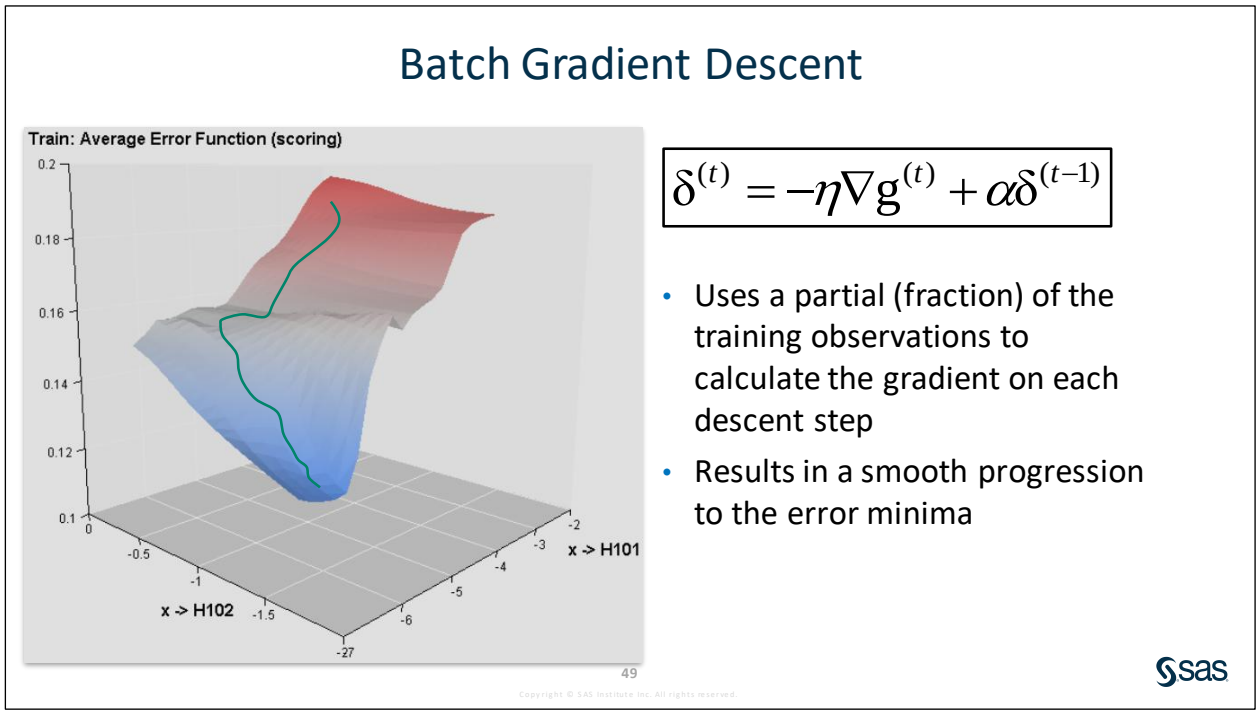
Copyright © SAS Institute Inc. All rights reserved.

The default optimization method in the Neural Network node when there are two or fewer hidden layers is a variant of the BFGS method known as *limited memory BFGS*. Like the original BFGS, the limited memory BFGS (L-BFGS) uses an estimation of the inverse Hessian to steer the search. But, whereas BFGS stores an *n* by *n* approximation to the Hessian (where *n* is the number of variables), the L-BFGS

variant stores only a few vectors that represent the approximation implicitly. Thus, L-BFGS is well suited for optimization problems with a large number of variables (Byrd et al. 1995).

The LBFGS is an optimization algorithm in the family of quasi-newton methods that approximates the BFGS algorithm using only some specific gradients to represent the approximation implicitly. It uses less computer memory due to its linear memory requirement. The algorithm starts with initial estimates of the optimal value of weights and progresses continuously to improve the estimates of the weights. The derivatives of the function of the estimates are used to drive the algorithm to find the direction of the steepest descent. The derivatives are also used to find the estimate of the Hessian matrix (second derivative).

Note: The default optimization method for the Neural Network node is *Automatic*. Automatic uses LBFGS if the network has two or fewer hidden layers. Otherwise, Stochastic Gradient Decent, discussed next, is used. LBFGS cannot be used in networks with more than two hidden layers.



Re-invented several times, the back propagation (*backprop*) algorithm initially used only *gradient descent* to determine an appropriate set of weights. The gradient, $\nabla \mathbf{g}^{(t)}$, is the vector of partial derivatives of the error function with respect to the weights. It points in the steepest direction uphill. By negating the step size (that is, *learning rate*) parameter, η, a step is made in the direction that is locally steepest downhill.

$$\boldsymbol{\delta}^{(t)} = -\eta \nabla \mathbf{g}^{(t)}$$

Unfortunately, as gradient descent approaches the desired weights, it exhibits numerous back-and-forth movements known as *hemstitching*. To control the training iterations wasted in this hemstitching, later versions of back propagation included a momentum term, yielding the modern update rule:

$$\boldsymbol{\delta}^{(t)} = -\eta \nabla \mathbf{g}^{(t)} + \alpha \boldsymbol{\delta}^{(t-1)}$$

The *momentum* term retains the last update vector, $\delta^{(t-1)}$, using this information to "dampen" potentially oscillating search paths. The cost is an extra learning rate parameter ($0 \leq \alpha \leq 1$) that must be set.

Note: The default value of α is 0. This means that, by default, backprop performs gradient descent.

In the (default) batch variant of the gradient descent algorithm, generation of the weight update vector is determined by using all of the examples in the training set. That is, the exact gradient is calculated, ensuring a relatively smooth progression to the error minima.

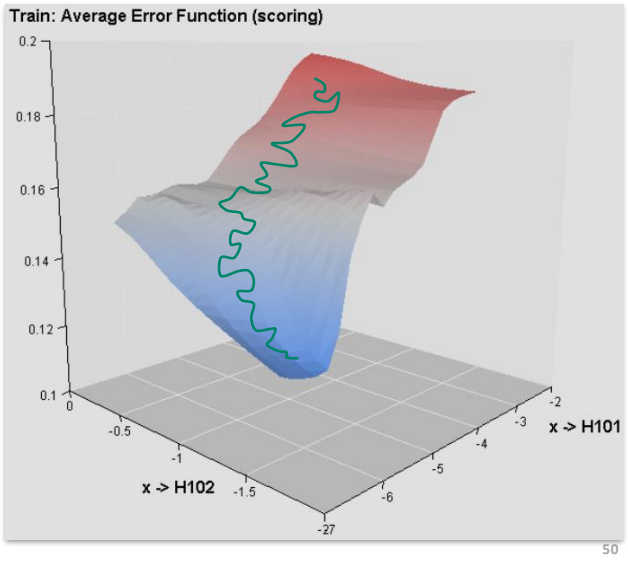
Note: For a linear neuron, with squared error, the error surface is a quadratic bowl. The vertical cross-sections are parabolas, and the horizontal cross-sections are ellipses. For multilayer networks, the error surface is much more complicated. But provided that the weights are not too big, locally the error surface can be well approximated by a piece of a quadratic bowl (Hinton 2013).

However, when the training data set is large, computing the exact gradient is computationally expensive. The entire training data set must be assessed on each step down the gradient. Moreover, if the data are redundant, the error gradient on the second half of the data will be almost identical to the gradient on the first half. In this event, it would be a waste of time to compute the gradient on the whole data set. You would be better off computing the gradient on a subset of the weights, updating the weights, and then repeating on a new subset. In this case, each weight update is based on an approximation to the true gradient. But as long as it points in approximately the same direction as the exact gradient, the approximate gradient is a useful alternative to computing the exact gradient (Hinton 2007).

A compromise between batch gradient descent and single-case stochastic gradient descent is to divide the training data into small batches, compute the gradient using a single batch, make an update, and then move to the next batch of observations. This is known as *mini-batch gradient descent*.

Like single-case stochastic gradient descent, mini-batch gradient descent is typically faster than batch gradient descent. And because the weights are updated less often than when single-observation stochastic gradient descent is used, mini-batch gradient descent typically uses less computation updating the weights, that is, each mini-batch computes the gradients for a number of cases in parallel. However, it is important that the mini-batches contain approximately balanced classes (Hinton 2013).

Stochastic Gradient Descent (SGD)



$$\delta^{(i)} = -\eta \nabla g^{(i)} + \alpha \delta^{(i-1)}$$

- Uses a single training observation to calculate an approximate gradient for each descent step
- Results in a chaotic progression to the error minima

50

Copyright © SAS Institute Inc. All rights reserved.

Stochastic gradient descent (SGD) is another numerical optimization method available for the Neural Network node in Model Studio.

Stochastic gradient descent is a stochastic approximation of the gradient descent optimization. It approximates the true gradient by using a single data point in the training data set.

The gradient descent is an optimization algorithm to find the minimum value for a function iteratively. It takes steps proportional to the negative of the gradient of the function at the current point. The gradient is a multi-variable generalization of the derivative. The derivative can be defined on functions of a single variable. For functions of several variables, which is the case of the predictive models, including the neural networks, the gradient is defined. As the gradient represent the slope of the tangent for a particular function, it points in the direction of the greatest rate of increase of the function, which in the neural network case would be the point that minimize the loss function.

If a multi-variable function ***F(x)*** is differentiable in a neighborhood of a point ***a***, ***F(x)*** decreases fastest if it goes from a point ***a*** in the direction of the negative gradient of ***F(x)*** at ***a***.

In Model Studio, the **Minibatch size property s**pecifies the size of the minibatches used for SGD optimization. The default value is 50.

Which optimization method should you use?

The LBFGS algorithm uses a limited amount of computer memory. It can often get the better solution than the SGD with fewer iterations. A strength of SGDs is that they are simple to implement and faster for problems that have many training examples and many predictor variables.

However, SGD methods have many disadvantages. One key disadvantage of SGDs is that they require much manual tuning of optimization parameters such as learning rates and convergence criteria. If you do not know the task at hand well, it is very difficult to find a good learning rate or a good convergence criterion. A standard strategy in this case is to run the learning algorithm with many optimization parameters and choose the model that gives the best performance on a validation set. Because you need to search over the large space of possible optimization parameters, this makes SGDs difficult to train in settings where running the optimization procedure many times is computationally expensive.

Note: For more details see “Alternatives to SGD: L-BFGS” at

https://raberrytv.wordpress.com/2015/06/20/alternatives-to-sgd-l-bfgs/

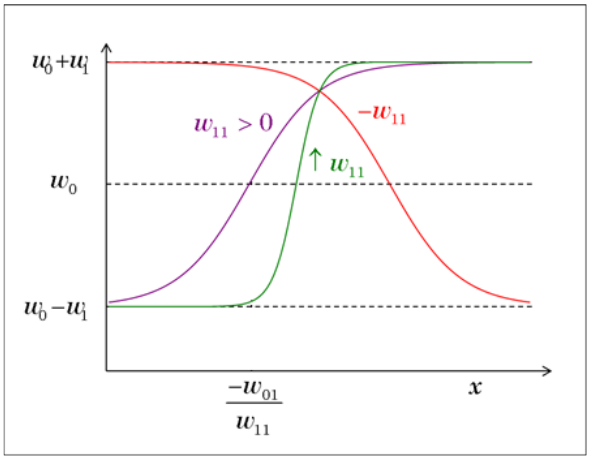
Shaping the Sigmoid

shape of the sigmoid

weights and biases

$w_0 + w_1 \tanh(w_{01} + w_{11}x)$

tanh Activation Function



Copyright © SAS Institute Inc. All rights reserved.

The weights and biases give the sigmoidal surfaces their range. For example, because the maximum value returned by the hyperbolic tangent activation function is 1, the upper bound of the hyperbolic tangent's activation range is given by the output unit's bias plus its weight:

$$w_0 + w_1 \tanh(w_{01} + w_{11}x) = w_0 + w_1(1) = w_0 + w_1$$

Conversely, the minimum activation value of the hyperbolic tangent function is –1, which means that the lower bound of the hyperbolic tangent's activation range is as follows:

$$w_0 + w_1 \tanh(w_{01} + w_{11}x) = w_0 + w_1(-1) = w_0 - w_1$$

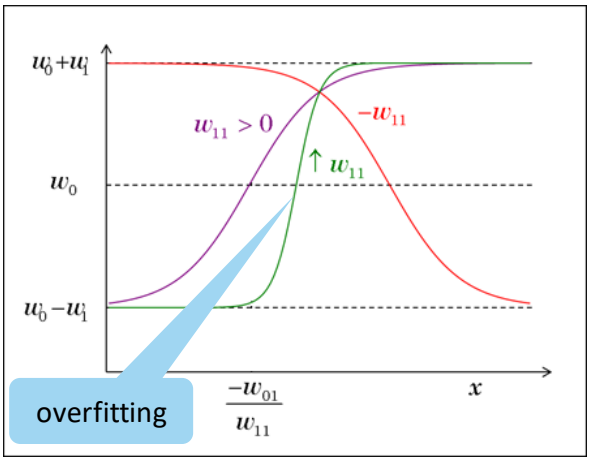
Shaping the Sigmoid

flexibility of the sigmoid

weights and biases

$w_0 + w_1 \tanh(w_{01} + w_{11}x)$

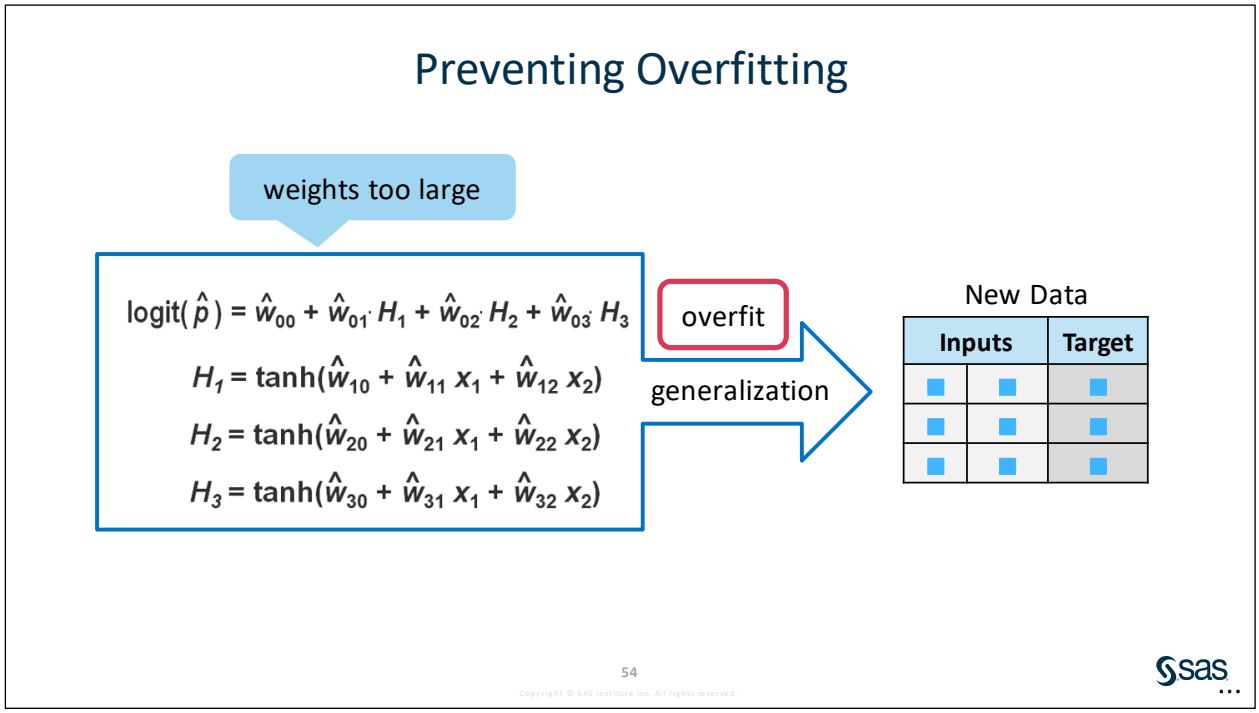
tanh Activation Function



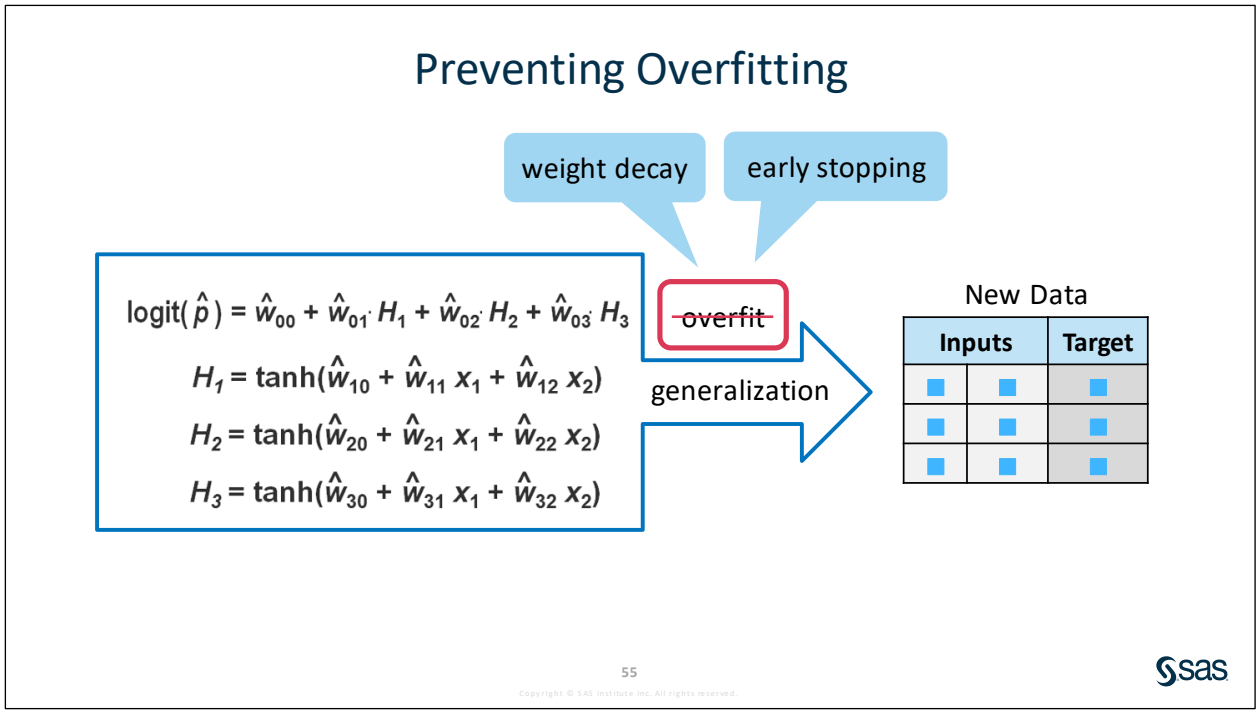
Copyright © SAS Institute Inc. All rights reserved.

The weights and biases also give the sigmoid surfaces their flexibility. The sign of the weight associated with input *x* controls the direction of the sigmoid. Positive weight values produce the familiar s-shaped curve, and a negative weight value flips the sigmoid horizontally. The larger the absolute value of the weight, the steeper the curve. ***Steep sigmoids are often held to be responsible for overfitting***.

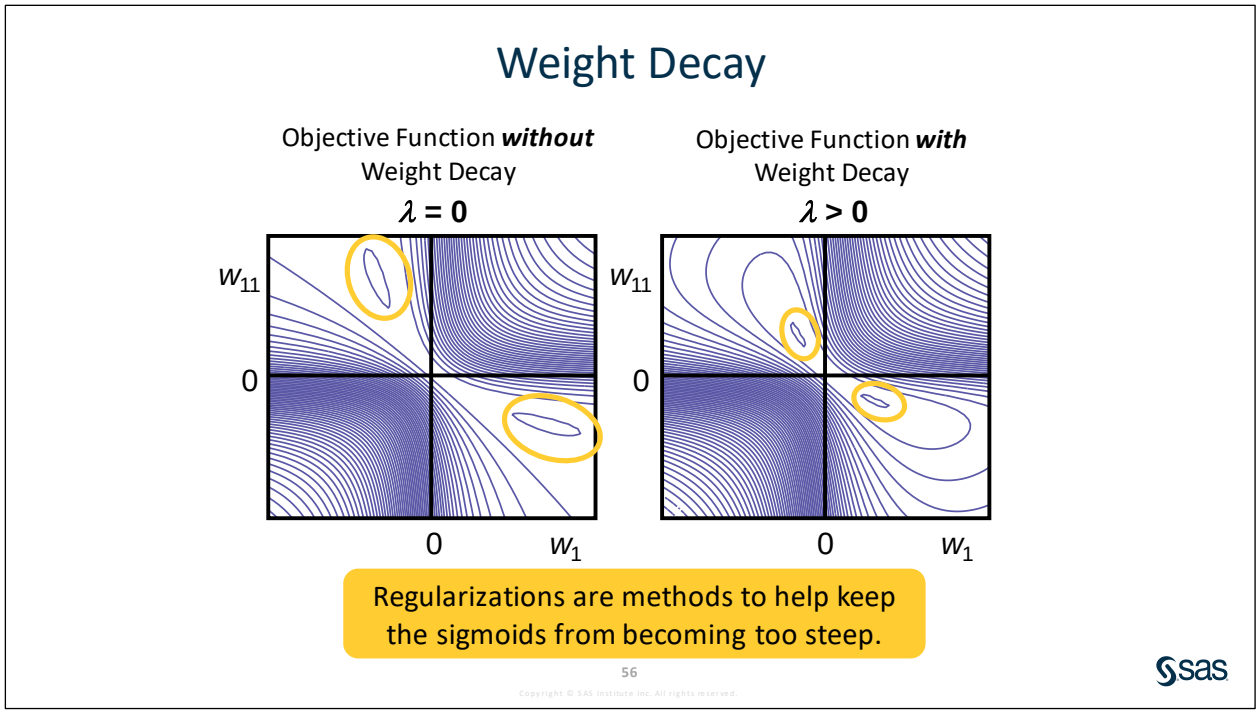
Note: Early stopping is one way to help keep the sigmoids from becoming too steep. Overfitting is controlled by the regularization terms L1 and L2 and the annealing rate, which you learn about later.



Recall that for neural networks, optimizing complexity is different compared to other models that we have seen thus far. Optimizing the complexity of a neural network model does not involve adding more terms (as for a regression model) or more rules (as for a decision tree). Here, it is the size of the weight estimates that are important in optimizing complexity.



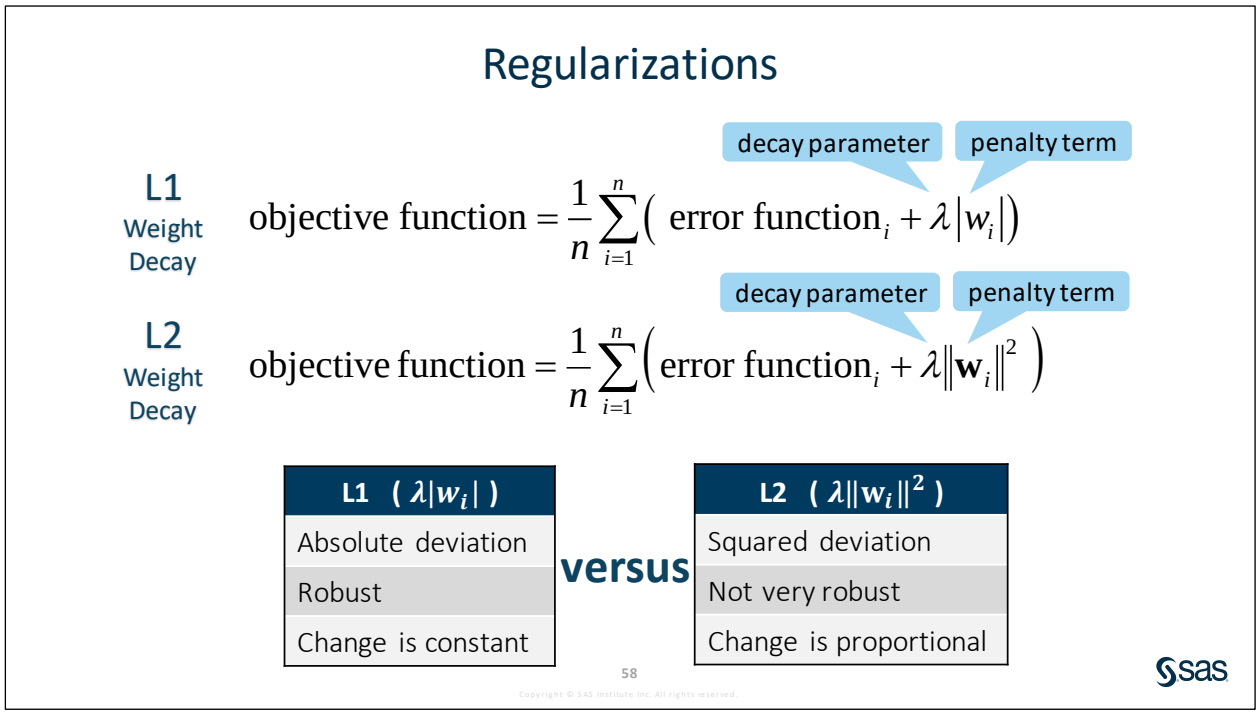
If the weights grow too large, the model will be overfit to the training data and not generalize well to new data. Weight decay and early stopping are the two primary methods to help avoid overfitting. These two methods are often used in tandem and are discussed next.



Bartlett (1997) demonstrated that generalization depends more on the magnitude of the weights than on the number of weights. Very large magnitude weights tend to generate an irregular fit to the data, as the model adapts to noise (random variation). In other words, large weights are responsible for overfitting.

The graph on the left shows the objective function without weight decay. The graph on the right shows the objective function with weight decay. In both graphs, the horizontal axis shows the magnitude of the weight w_1, and the vertical axis shows the magnitude of the weight w_{11}.

In the graph on the left, with no weight decay, the minima are farther from the origin, so the weights used to find the minima are large. In the graph on the right, the minima are constrained closer to the origin, so the weights used to find the minima are smaller (and thus, less likely to overfit).



Weight decay is one way to keep the weights from growing too large. L1 regularization is similar to L2 regularization in that both methods penalize the objective function for large network weights. The two regularizations differ in that L2 regularization penalizes the objective function by an amount proportional to the weight size, whereas L1's penalty is constant relative to changes in the weights. This means that L2 is likely to penalize larger weights to a greater degree than L1, but to a lesser degree than L1 when the weights are small. L1 regularization encourages sparsity because it is found to "concentrate the weight of the network in a relatively small number of high-importance connections, while the other weights are driven toward zero" (Nielsen 2015). L2 regularization is viewed as shrinking weights toward zero and can be considered not as robust as L1 regularization due to the large penalty that can arise from outliers.

The decay parameter λ, which can range from 0 to 1, controls the relative importance of the penalty term. Specifying too large of a penalty term risks the model underfitting the data. However, as Ripley (1996) points out, the advantages of weight decay far outweigh its risks [emphasis added]:

> Weight decay helps the optimization in several ways. When weight decay terms are included, it is normal to find fewer local minima, and as the objective function is more nearly quadratic, the quasi-Newton and conjugate gradient methods exhibit super-linear convergence in many fewer iterations. ***There seems no reason to ever exclude a regularizer such as weight decay***. (pp. 159-160)

In any event, the influence of the penalty term is usually kept extremely small. In many published studies, the magnitude of the decay parameter (λ) is on the order of 0.000001.

Improving a Neural Network Model by Changing the Network Learning and Optimization Parameters

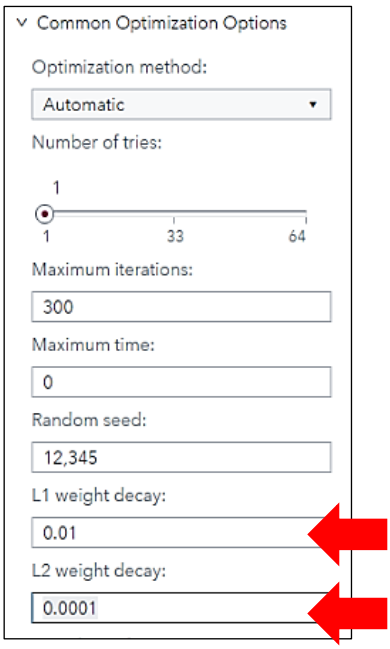
In this demonstration, you change the previous settings of the Neural Network node in the Lesson 4 pipeline. You modify the learning and optimization parameters and compare this model performance to the other model in the pipeline.

1. Recall that the previous model, based on changes in the network architecture, achieved an average squared error of 0. 0720 on the VALIDATE partition. This fit statistic showed quite an improvement over the first model built by using the default settings.

   Try to improve the neural network performance by changing now some of the default settings assigned to the learning and optimization parameters.

2. Under the Common Optimization Options properties, increase **L1 weight decay** from 0 to **0.01**.

3. Decrease **L2 weight decay** from 0.1 to **0.0001**.



   In L1, we penalize the absolute value of the weights. Unlike L2, the weights can be reduced to zero here. Hence, it is very useful when we are trying to compress our model. Otherwise, we usually prefer L2 over it.

   Mathematically speaking, weight decay adds a regularization term in order to prevent the coefficients to fit so perfectly as to overfit. The difference between L1 and L2 is that L2 is the sum of the square of the weights, whereas L1 is just the sum of the weights.

4. Run the **Neural Network** node.

5. Open the Results window for the node.

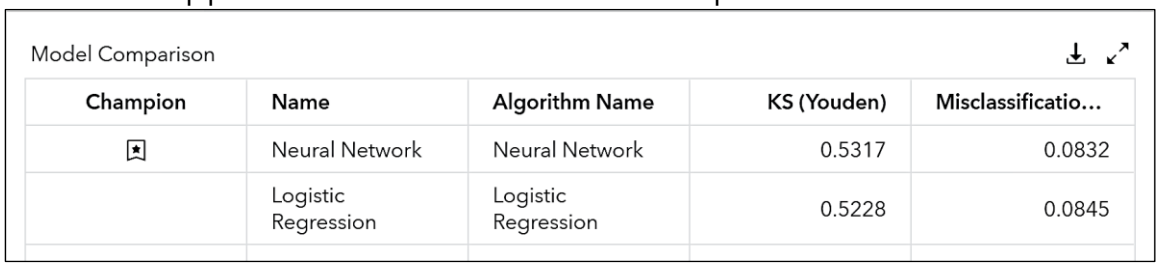
6. Click the **Assessment** tab.

Target ...	Data Role	Partitio...	Formatt...	Sum of ...	Averag...
churn	TRAIN	1	1	39,590	0.0664
churn	VALIDATE	0	0	16,967	0.0699

   The average squared error for the tuned neural network model is 0.0699 on the VALIDATE partition. This fit statistic is slightly better than the previous model trained in the last demonstration.

7. Close the Results window.

8. Run the entire pipeline and view the results of model comparison.



Model Comparison

Champion	Name	Algorithm Name	KS (Youden)	Misclassificatio...
★	Neural Network	Neural Network	0.5317	0.0832
	Logistic Regression	Logistic Regression	0.5228	0.0845

The neural network model is the champion of the pipeline based on default KS(Youden) statistic.

9. Close the Results window.

End of Demonstration